CAMPUSNOTE PRO

REQUIREMENTS ELICITATION AND ANALYSIS DOCUMENT

Student Number	Name and Surname	Role	Photo
220303008	Pelin Zeynep Kaya	Leader	
220303012	Arjin İlhan	Developer	
220303014	Mustafa Baver Çalış	Developer	
220303006	Hüseyin Fidan	Tester	

Document Name	: CampusNotePro_READ.docx
Version No	: 3.0
Version Date	: 19.05.2026

DOCUMENT APPROVAL INFORMATION

		Sign
Contributors	Pelin Zeynep Kaya (Leader) Hüseyin Fidan (Tester) *Arjin İlhan (Developer)* *Mustafa Baver Çalış (Developer)*	☑ ☑ ☑ ☑
Reviewer-1 (LLM)	Gemini Pro 3.1	☐
Reviewer-2 (LLM)	NotebookLM	☐
Reviewer-3	Arjin İlhan	☑
Reviewer-4	Mustafa Baver Çalış	☑
Reviewer-5	Hüseyin Fidan	☑
Quality Manager		☐
Stakeholder-1		☐
Stakeholder (Instructor)	Haluk Gümüşkaya	☐

CHANGE RECORDS

Version	Comments	Date
1	**Initial Draft**	22/04/2026
2	**Peer Review & Analysis Refinement**	28/04/2026
3	**Final Validation & Traceability Matrix**	04/05/2026

DISTRIBUTION RECORDS

Distribution No	Version No	Comments	Distribution Method
			Upload to Google Classroom

Contents

1. INTRODUCTION

1.1. Objectives of Requirements Engineering

Requirements Engineering (RE) is the systematic process of discovering, documenting, and maintaining the specific needs of a system's users. For CampusNote Pro, this discipline is vital because it ensures the platform effectively solves the problem of information loss students face due to fragmented communication channels while incentivizing active participation. By utilizing elicitation to gather student needs, analysis to structure those needs, validation to ensure the system is realistic, and management to track progress, we establish a professional foundation that avoids the superficiality of typical academic projects. Furthermore, by utilizing gamification, we aim to transform the archiving process into a rewarding experience, encouraging students to provide high-quality materials in exchange for academic rank and rewards.

Key activities within this process include:

- **Requirements Elicitation:** Collecting needs from students and faculty to understand the academic hierarchy and identifying pain points in existing sharing behaviors.
- **Requirements Analysis:** Refining gathered needs into technical models like Use Case and Data Diagrams to ensure structural integrity.
- **Validation:** Ensuring the requirements are realistic, consistent, and testable before implementation begins.
- **Requirements Management:** Maintaining a Traceability Matrix to link every student need to a specific system feature, reward trigger, or test case.

1.2. Purpose of the Document

The primary purpose of this Requirements Elicitation and Analysis Document (READ) is to formally initiate the development process for CampusNote Pro. This document establishes a strong foundation for the system's design by detailing student-oriented needs and organizational artifacts. It serves as a synchronized roadmap for the project leader, developers, and testers to ensure the system remains consistent throughout the 10-week development period.

1.3. Project Scope

The scope focuses on a centralized, hierarchical platform that empowers students to archive and verify study materials.

In-Scope:

- A structured repository for student notes indexed by Faculty and Course Code.
- A verification system driven by an Academic Ranking Algorithm.
- Gamification Engine: User Ranking system and "CampusCoin" distribution.
- Role-Based Access Control and a responsive web interface.

Out-of-Scope:

- Development of native mobile applications for iOS or Android.
- Built-in tools for creating or editing documents within the platform.
- Marketplace features for financial transactions between students.

1.4. Objectives and Success Criteria

The ultimate goal of CampusNote Pro is to transform academic collaboration into a traceable and verified knowledge ecosystem for the student body. We aim to provide a reliable environment where academic materials move through a transparent lifecycle from "Draft" to "Published" status.

The success of the system will be measured by its ability to maintain high availability and deliver document rendering times of less than 2 seconds, especially during peak exam periods. Furthermore, 100% of the core student requirements must be linked to automated verification tests to guarantee the platform's reliability.

Table 1.1. Objectives and measurable success criteria.

Objective	Measurable Success Criteria
System Performance	The system shall deliver document preview and rendering times of ≤ 2.0 seconds during peak traffic periods.
Platform Reliability	The system shall maintain an operational cloud availability (uptime) of at least 99.9%.
Content Quality	The platform shall maintain a global average AI Verification Score of ≥ 80/100 across all submitted documents.
Technical Traceability	100% of documented Functional Requirements shall be bi-directionally traceable to automated unit and integration tests.
User Motivation (Gamification)	At least 30% of users who upload a document will subsequently upload a second document to earn additional CampusCoins.
Data Security	The system shall experience 0 unauthorized privilege escalation incidents to the Administrative Dashboard.

1.5. Stakeholders

Table 1.2. Stakeholder list and expectations.

Stakeholder	Role	Importance	Expectations
Student (Uploader)	Primary User / Contributor	Critical	Easy document upload process, accurate categorization (Faculty/Course Code), and earning CampusCoins/virtual ranks through successful AI verification.
Student (Consumer)	Primary User / Consumer	Critical	Fast access to high-quality notes, document rendering in ≤ 2.0 seconds, and a reliable search/filter system.
System Administrator	Moderator / Operations	Critical	A secure dashboard to manage flagged documents, monitor cloud storage capacity, and suspend accounts violating academic integrity.
Course Professor	Client / Evaluator	Critical	Professional documentation (READ), adherence to SWEBOK v4.0 standards, and 100% consistency between UML models and text.
Project Team	Developers & Managers	Critical	A clear technical roadmap, established bi-directional traceability, and successful system deployment on Heroku.
Istanbul Arel University	Institutional Host	Major	Preservation of academic knowledge and the elimination of information silos across social media groups.

1.6. System Context

CampusNote Pro operates as a centralized, web-based Software-as-a-Service (SaaS) platform hosted on the Heroku cloud infrastructure. The system is designed with a strict client-server architecture, assuming a stable internet connection for REST API consumption.

The application establishes clear architectural boundaries and interfaces with the following distinct environments:

- **Client Interface:** End-users interact with the system via standard modern web browsers using a responsive frontend styled with Tailwind CSS.
- **Application Server:** The core business logic, gamification engine, and API routing are handled by a Java Spring Boot backend deployed on Heroku.
- **Relational Persistence:** A PostgreSQL database handles the persistent storage of structured academic metadata, user credentials, and gamification telemetry (e.g., CampusCoins, virtual ranks).
- **Object Storage (External):** Amazon Web Services (AWS) S3 is utilized exclusively for the secure physical storage and retrieval of uploaded PDF document files.
- **AI Verification Engine (External):** The system continuously communicates with an external AI Ranking Service—developed in **Python** using the **PyTorch** machine learning framework—via REST API to extract text, calculate keyword density, and return an automated 0-100 quality score for document moderation.

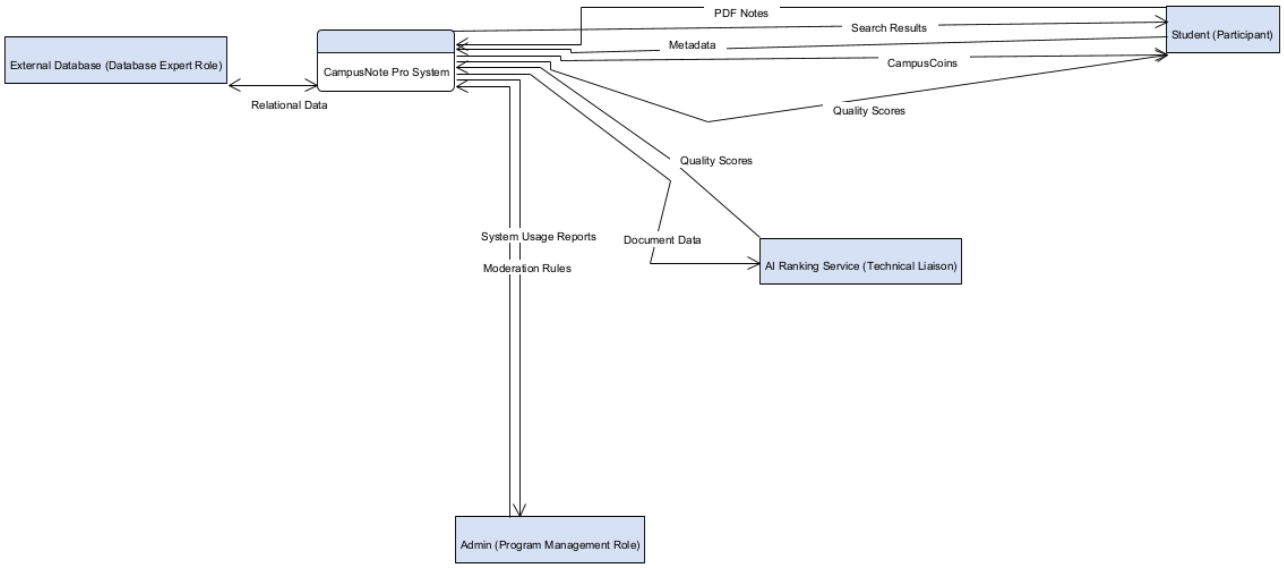


***Figure 1.1**. System Context Diagram*

2. REQUIREMENTS ELICITATION

This section documents the primary needs of the CampusNote Pro platform, derived from stakeholder analysis and the specific academic context of Istanbul Arel University.

2.1. Elicitation Methods and Tools

The following methods were used to capture requirements during the development cycle:

- **Existing System Analysis:** Analysis of fragmented sharing behavior within WhatsApp and Telegram groups to identify common data loss points. Our analysis identified three critical information loss patterns:
  - **Temporal Decay:** Crucial exam notes are pushed out of view by daily chat volume within 48 hours, making them difficult to retrieve.
  - **Metadata Absence:** Files are shared without clear Course Codes or Department labels, rendering standard keyword search ineffective.

    - **The Trust Gap:** The absence of a verification mechanism forces students to manually cross-check PDF accuracy, leading to wasted study time.
- **Scenario-Based Elicitation:** Creation of narrative interaction paths to determine how users navigate the academic hierarchy.
- **Proxy Stakeholder Brainstorming:** The internal project team acted as proxy students to define core pain points regarding resource accessibility.

Traceability and Mapping Rule: To ensure high-level bi-directional traceability and simplify modeling in Visual Paradigm, the textual specification is strictly decoupled from the Use Case numbering. Instead, every unique Requirement ID (e.g., FR-ST-33) must be tagged directly onto its corresponding conceptual element (Use Case bubble, Database Column, or Activity Node) within the UML diagrams. This guarantees that no architectural model is drawn without a justifying business requirement.

2.2. High-Level Requirements Summary

The table below provides a high-level summary of the essential requirements that form the backbone of the CampusNote Pro architecture.

Table 2.1. *High-level requirements summary.*

ID	Description	Type
FR-ST-15	The system shall restrict study note uploads exclusively to the PDF file format.	Functional
FR-DOC-17	The system shall categorize documents based on the official Istanbul Arel University Faculty hierarchy.	Functional
FR-DOC-28	The system shall transition document status to "Published" only after assigned AI quality scores meet the passing threshold.	Functional
FR-ST-44	The system shall initiate a secure PDF download upon a user selecting the retrieval icon for a published document.	Functional
NFR-DOC-65	The system shall maintain an operational cloud availability (uptime) of at least 99.9%.	Non-Functional
NFR-ST-69	The system shall generate and render document previews in ≤ 2.0 seconds.	Non-Functional
CON-TECH-74	The system shall utilize the Heroku cloud platform for final production hosting.	Constraint
CON-TECH-76	The system shall be delivered as a responsive web SaaS, excluding native mobile application development.	Constraint

2.3. Functional Requirements

Table 2.2. Identity & Access Management

ID	Name	Description	Priority
FR-ST-01	User Authentication	The system shall authenticate users using an "@arel.edu.tr" email domain.	Must Have

FR-ST-02	Invalid Credentials	The system shall deny login attempts containing incorrect credentials.	Must Have
FR-ST-03	Login Error	The system shall display an error message upon a denied login attempt.	Must Have
FR-ST-04	Password Recovery	The system shall provide a password reset link via email.	Should Have
FR-ST-05	Session Termination	The system shall terminate the active session upon clicking the sign-out button.	Must Have

Table 2.3. Dashboard & User Analytics

ID	Name	Description	Priority
FR-ST-06	Department Display	The system shall display the user's enrolled academic Department on the dashboard navigation bar.	Should Have
FR-ST-07	Academic Year Display	The system shall display the user's current academic year on the dashboard navigation bar.	Should Have
FR-ST-08	Registration Date	The system shall display the user's account creation date on their profile.	Could Have
FR-ST-09	Upload Analytics	The system shall display the total count of documents uploaded by the user.	Should Have
FR-ST-10	Download Analytics	The system shall display the aggregated total of downloads received across all of the user's documents.	Should Have
FR-ST-11	Upload History	The system shall display a chronological list of the user's previously uploaded documents.	Must Have
FR-ST-12	History Status	The system shall display the current verification status (e.g., Draft, Published) alongside each document in the upload history list.	Must Have
FR-ST-13	Like Analytics	The system shall display the aggregated total of "Likes" received across all of the user's documents on their profile.	Should Have
FR-ST-14	Theme Toggle	The system shall permit users to toggle the interface rendering between Light and Dark visual themes.	Could Have

Table 2.4. Document Ingestion Subsystem

ID	Name	Description	Priority
FR-ST-15	PDF Enforcement	The system shall allow students to upload documents strictly in PDF format.	Must Have
FR-ST-16	Size Restriction	The system shall reject uploaded files exceeding 20MB.	Should Have
FR-DOC-17	Faculty Classification	The system shall categorize documents based on the official Arel University Faculty list.	Must Have
FR-DOC-18	Department Filtering	The system shall filter available Department options based on the previously selected Faculty.	Must Have
FR-ST-19	Course Code Entry	The system shall require the input of a valid Course Code prior to document submission.	Must Have
FR-ST-20	Initial Status	The system shall assign a "Draft" status to a document immediately upon successful upload.	Must Have
FR-ST-21	Submission Validation	The system shall disable the "Upload Document" button until a valid PDF file is provided and all mandatory categorization fields are selected.	Must Have
FR-ST-22	Upload Cancellation	The system shall abort the upload process and close the modal upon the user clicking the "Cancel" button.	Could Have

Table 2.5. AI Academic Ranking Algorithm

ID	Name	Description	Priority
FR-ST-23	AI Service Trigger	The system shall transmit the "Draft" PDF document to the external AI Ranking Service.	Must Have
FR-ST-24	Processing Status	The system shall assign an "Under Review" status to documents that are actively being processed by the AI Ranking Service.	Must Have

FR-ST-25	Text Extraction	The system shall extract text content from the submitted PDF document.	Must Have
FR-ST-26	Keyword Comparison	The system shall compare extracted text against the course-specific academic keyword dictionary.	Must Have
FR-ST-27	Quality Scoring	The system shall calculate an integer quality score ranging from 0 to 100 based on keyword density.	Must Have
FR-DOC-28	Publication Rule	The system shall change the document status to "Published" if the score is greater than or equal to the passing threshold.	Must Have
FR-DOC-29	AI Rejection Routing	The system shall change the document status to "Flagged" and route it to the Moderation Queue if the AI quality score is strictly less than the passing threshold.	Must Have
FR-ST-30	Verification Notice	The system shall notify the uploader regarding the final AI verification status.	Should Have

Table 2.6. Gamification & Engagement

ID	Name	Description	Priority
FR-ST-31	CampusCoin Display	The system shall display the user's total CampusCoin balance on the dashboard.	Should Have
FR-ST-32	Rank Display	The system shall display the user's current virtual rank on the dashboard.	Could Have
FR-ST-33	Reward Calculation	The system shall calculate the document's CampusCoin reward by multiplying its associated Course's credit value (AKTS) by 10.	Must Have
FR-ST-34	Coin Distribution	The system shall add the calculated CampusCoin reward to the uploader's account balance immediately upon the document achieving "Published" status.	Must Have
FR-ST-35	Rank Evaluation	The system shall evaluate the user's total CampusCoin balance against the virtual rank thresholds.	Should Have
FR-ST-36	Rank Upgrade	The system shall upgrade the user's virtual rank when their coin balance meets the next configured threshold.	Could Have
FR-ST-37	Document Liking	The system shall permit users to cast a "Like" on a document possessing a "Published" status.	Should Have
FR-ST-38	Leaderboard Display	The system shall display a globally ranked leaderboard of users, sorted descending by their total CampusCoin balance.	Should Have

Table 2.7. Search, Retrieval, and Navigation

ID	Name	Description	Priority
FR-ST-39	Keyword Input	The system shall provide a text input field for keyword-based document searching.	Must Have
FR-ST-40	Verified Filter	The system shall restrict search results exclusively to documents possessing a "Published" status.	Must Have
FR-ST-41	Faculty Filter	The system shall filter search results based on a selected Faculty parameter.	Should Have
FR-ST-42	Result Sorting	The system shall sort search results by the highest download count.	Should Have
FR-ST-43	Document Preview	The system shall display a thumbnail preview consisting of the first page of the PDF.	Could Have
FR-ST-44	File Retrieval	The system shall initiate a PDF file download upon clicking the download icon.	Must Have
FR-ST-45	Download Tracking	The system shall increment the document's download counter by one upon a successful retrieval.	Could Have
FR-ST-46	View Tracking	The system shall increment the document's view counter by one each time a user renders the document's preview thumbnail.	Could Have

Table 2.8. Administrative Operations

ID	Name	Description	Priority
FR-ST-47	Restricted Access	The system shall deny access to the Admin Dashboard for any user who does not possess the Administrator role.	Must Have

FR-ST-48	User Listing	The system shall display a data table of registered student accounts.	Should Have
FR-ST-49	Admin User Search	The system shall provide a text input field within the Admin Dashboard for administrators to filter the user table by name or email address.	Should Have
FR-ST-50	Account Suspension	The system shall permit administrators to change a user's account status to "Suspended".	Should Have
FR-ST-51	Suspension Enforcement	The system shall deny login capabilities to "Suspended" accounts.	Must Have
FR-ST-52	Moderation Queue	The system shall display a list of all documents possessing a "Flagged" status within the Admin Dashboard.	Must Have
FR-ST-53	Moderation Metric	The system shall display the aggregated total count of currently "Flagged" documents on the Admin overview dashboard.	Could Have
FR-ST-54	Moderation Context	The system shall display the document's calculated AI Quality Score alongside its entry in the moderation queue.	Should Have
FR-ST-55	Content Deletion	The system shall permit administrators to delete documents from the repository.	Must Have
FR-ST-56	Flag Dismissal	The system shall permit administrators to dismiss reports on a flagged document, removing it from the moderation queue while retaining its "Published" status.	Must Have

Table 2.9. System Operations

ID	Name	Description	Priority
FR-ST-57	Automated Flagging	The system shall change a document's status to "Flagged" when it accumulates 5 negative user reports.	Could Have
FR-ST-58	Audit Logging: Deletion	The system shall write a timestamped log entry to the audit file immediately upon an administrator deleting a document.	Should Have
FR-ST-59	Audit Logging: Suspension	The system shall write a timestamped log entry to the audit file immediately upon an administrator suspending a user account.	Should Have
FR-ST-60	Audit Logging: Configuration	The system shall write a timestamped log entry to the audit file immediately upon an administrator modifying the global AI passing threshold.	Should Have
FR-ST-61	Audit Logging: Dismissal	The system shall write a timestamped log entry to the audit file immediately upon an administrator dismissing a document report.	Should Have
FR-ST-62	Audit Trail Display	The system shall display a chronological list of recent administrative log entries on the Admin Dashboard.	Should Have
FR-ST-63	Storage Telemetry	The system shall display the total database storage consumption in gigabytes (GB) on the Admin Dashboard.	Could Have
FR-DOC-64	Threshold Configuration	The system shall permit the Administrator role to modify the global AI algorithm passing score.	Should Have

2.4. Non-Functional Requirements

2.4.1. Specific Quality Requirements

Quality attributes define the non-behavioral aspects of the system, such as reliability and security, which are essential for maintaining user trust in an academic environment.

Table 2.8. Specific quality requirements.

ID	Name	Description	Priority
NFR-DOC-65	Operational Uptime	The system shall maintain an operational availability of at least 99.9%.	Must Have

NFR-ST-66	Verification Integrity	The system shall ensure 100% of "Published" notes pass the Ranking Algorithm.	Must Have
NFR-ST-67	Password Security	The system shall encrypt 100% of user passwords using BCrypt hashing.	Must Have
NFR-ST-68	Data Persistence	The system shall perform automated database backups every 24 hours.	Should Have

2.4.2. Performance Requirements

Performance requirements define the speed and capacity constraints. These metrics are critical during high-traffic periods, such as final exam weeks.

Table 2.4. Performance requirements.

ID	Name	Description	Priority
NFR-ST-69	Preview Latency	The system shall render document previews within 2.0 seconds.	Should Have
NFR-ST-70	Search Response	The system shall return search results within 500 milliseconds.	Must Have
NFR-ST-71	Algorithm Throughput	The system shall process the ranking analysis in under 5.0 seconds per 10MB file.	Must Have

2.4.3. Constraints

Performance requirements define the speed and capacity constraints. These metrics are critical during high-traffic periods, such as final exam weeks.

Table 2.5. Constraints.

ID	Name	Description	Priority
CON-TECH-72	Development Framework	The system shall be developed using the Java Spring Boot framework.	Must Have
CON-TECH-73	Persistence Layer	The system shall utilize a PostgreSQL relational database.	Must Have
CON-TECH-74	Hosting Infrastructure	The system shall be deployed on the Render cloud platform.	Must Have
CON-TECH-75	Interface Styling	The system shall implement user interfaces using Tailwind CSS.	Should Have
CON-TECH-76	Deployment Boundary	The system shall exclude native mobile application development.	Won't Have
CON-TECH-77	Object Storage	The system shall utilize Amazon Web Services (AWS) S3 for the physical storage of uploaded PDF document files.	Must Have

2.5. Assumptions

Assumptions are conditions considered to be true for the purpose of the requirement analysis, acknowledging factors outside the direct control of the project team.

Table 2.6. Assumptions.

ID	Name	Description

ASM-ENV-78	Connectivity	It is assumed that users possess stable internet connectivity for cloud platform access.
ASM-DOC-79	University Hierarchy	It is assumed that the university provides an up-to-date hierarchy of all faculties and departments.
ASM-ST-80	User Motivation	It is assumed that students are motivated to upload high-quality notes by virtual ranks and CampusCoins.
ASM-TECH-81	Resource Availability	It is assumed that the Heroku cloud platform provides sufficient resources to support document rendering times.
ASM-ST-82	Algorithmic Autonomy	It is assumed that the Ranking Algorithm is capable of evaluating documents without manual human intervention.

3. REQUIREMENTS ANALYSIS

3.1. Scenarios

The following scenarios describe realistic interactions between users and the system to illustrate how the requirements solve real-world problems.

Scenario 1: Successful Document Upload

- **Actor**: Student (Uploader)
- **Goal**: Share a high-quality study note with peers to earn CampusCoins and improve their academic rank
- **Steps:**
  1. Login: Student authenticates using their @arel.edu.tr email. (FR-ST-01)
  2. Navigation: Student selects "Faculty of Engineering" and "Course: CS101." (FR-DOC-09)
  3. Upload: Student uploads a PDF summary of the course. (FR-ST-07)
  4. Analysis: The system automatically triggers the Academic Ranking Algorithm. (FR-ST-13)
  5. Scoring: The algorithm scans for keywords and assigns a quality score of 85/100. (FR-ST-15)
  6. Publication: Since 85 exceeds the threshold, the system changes the status to "Published." (FR-ST-16)
  7. Reward: The system deposits 10 CampusCoins into the Student's profile. (FR-ST-17)

Scenario 2: Low-Quality Content Rejection

- **Actor:** Student (Uploader)
- **Goal:** Attempt to upload a low-quality, corrupted, or irrelevant file.
- **Steps:**
  1. **Upload:** Student uploads a PDF file containing only one sentence. (**FR-ST-07**)
  2. **Analysis:** The system executes the Academic Ranking Algorithm. (**FR-ST-13**)
  3. **Failure:** The algorithm detects a "Low Keyword Density" and assigns a score of 10/100. (**FR-ST-15**)
  4. **Rejection:** The system retains the file in "Draft" status and does not publish it. (**FR-ST-16**)
  5. **Notification:** The system notifies the user that the document failed verification.
  6. **No Reward:** The user's CampusCoin balance remains unchanged.

Scenario 3: Hierarchical Keyword Search and Download

- **Actor:** Student (Consumer)
- **Goal:** Find and download verified materials for an exam.
- **Steps:**
    1. **Search:** Student enters "Calculus 1" into the search bar. (**FR-ST-19**)
    2. **Filter:** Student filters the results to only show "Faculty of Engineering." (**FR-ST-20**)
    3. **Selection:** Student sorts the results by "Most Downloaded" to find the best note. (**FR-ST-21**)
    4. **Preview:** Student views the thumbnail preview of the first page. (**FR-ST-22**)
    5. **Download:** Student clicks download; the system serves the PDF and increments the download counter. (**FR-ST-12**)

3.2. Use Case Models

The detailed use case descriptions below provide the logic and flow for technical implementation, ensuring a clear understanding of system-actor interactions.

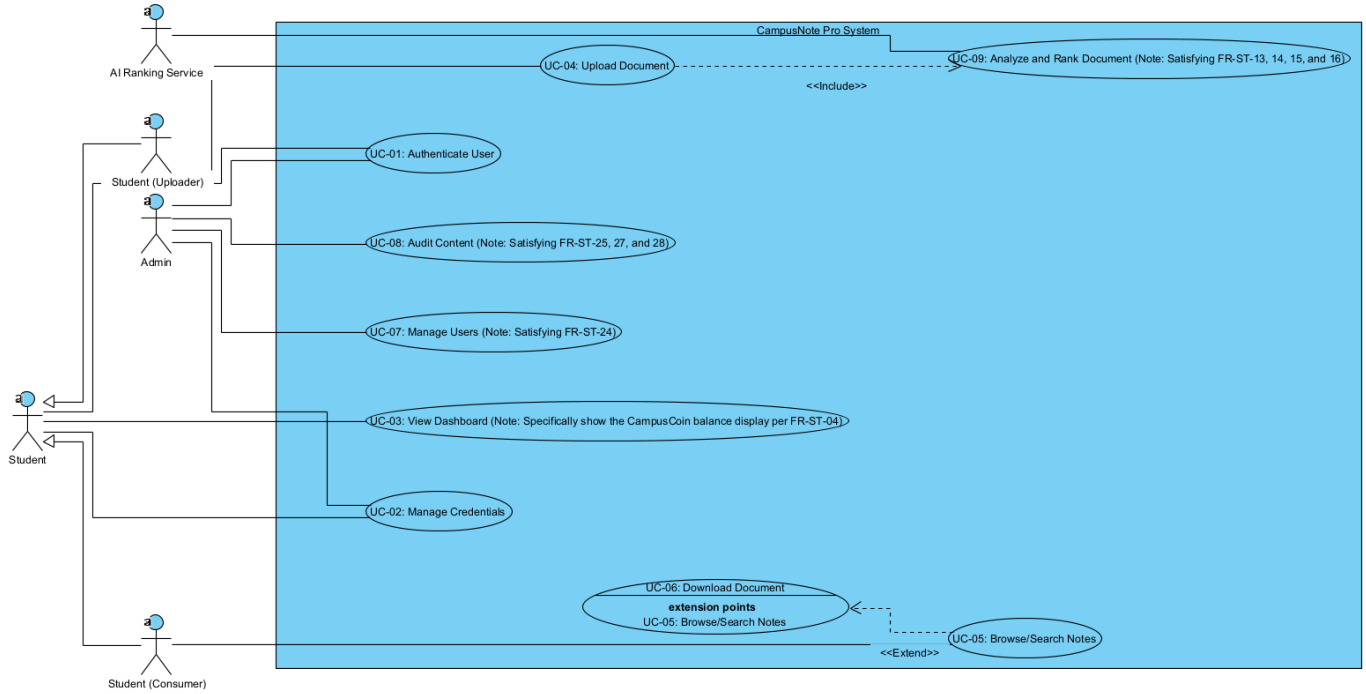


Figure 3.1. *Use Case Diagram.*

Use Case 1: UC-04 Authenticate User

Table 3.1. *Use case description for UC-04 Authenticate User.*

Field	Description
Use Case Name	Authenticate User
Actor(s)	Student, Instructor, Admin
Description	The user logs into the system using university credentials.
Preconditions	The user has an active university account.
Main Flow	1. User enters username. 2. User enters password. 3. System validates credentials. 4. System grants access.
Alternative Flow	1. Invalid credentials entered; system displays error message.
Postconditions	User is logged in and redirected to the dashboard.

Use Case 2: UC-05 Upload Document

Table 3.2. Use case description for UC-05 Upload Document.

Field	Description
Use Case Name	UC-05 Upload Document
Actor(s)	Student (Uploader)
Description	The user uploads a PDF study note and categorizes it strictly by Faculty, Department, and Course Code.
Preconditions	The user is successfully authenticated into the system.
Main Flow	1. User selects Upload. 2. User selects a PDF file. 3. User categorizes by Faculty and Course. 4. User confirms upload. 5. System saves file as Draft.
Alternative Flow	1. If file is not PDF, system rejects upload and displays error.
Postconditions	Document is stored in Draft status pending AI verification.

Use Case 3: UC-07 Verify, Publish and Reward

Table 3.3. Use case description for UC-07 Verify, Publish and Reward.

Field	Description
Use Case Name	UC-07 Verify, Publish and Reward
Actor(s)	AI Ranking Algorithm (System)
Description	The AI evaluates a draft document, publishes it if score is above 80, and awards CampusCoins.
Preconditions	A document exists with Draft status.
Main Flow	1. AI algorithm evaluates document. 2. Algorithm assigns quality score. 3. System checks if score is passing. 4. System changes status to Published.

	5. System awards CampusCoins to uploader.
Alternative Flow	1. If score is failing, system rejects document and leaves it as Draft.
Postconditions	Document is published and user is rewarded, or it is rejected.

Use Case 4: UC-08 Keyword Search and Download

Table 3.4. Use case description for UC-08 Keyword Search and Download.

Field	Description
Use Case Name	UC-08 Keyword Search and Download
Actor(s)	Student (Consumer)
Description	The user searches for published notes using keywords and downloads a verified file.
Preconditions	User is authenticated and published documents exist.
Main Flow	1. User enters keyword. 2. System searches metadata. 3. System displays matching documents. 4. User selects a document to download. 5. System serves the PDF file.
Alternative Flow	1. If no matches found, system displays “No Results Found” message.
Postconditions	User successfully downloads the document.

3.3. Data Models (Entity-Relationship (ER Diagrams))

The Data Model for CampusNote Pro represents the hierarchical structure of Istanbul Arel University's academic departments and the lifecycle of shared academic documents.

3.3.1. Entity Descriptions and Attributes

Table 3.5. Entity descriptions and database attributes

Entity	Attributes	Primary Key (PK)	Foreign Key (FK)
User	userId, fullName, email, passwordHash, role, coinBalance, rank	userId	None
Faculty	facultyId, facultyName	facultyId	None
Department	departmentId, departmentName	departmentId	facultyId

Course	courseId, courseCode, courseName	courseId	departmentId
Document	docId, title, filePath, status, uploadDate	docId	userId, courseId
RankingResult	rankingId, score, feedback, verifiedAt	rankingId	docId
Flag	flagId, reason, status, createdAt	flagId	userId, docId

3.3.2. Relationships

- **Faculty - Department:** One Faculty contains many Departments (1:N).
- **Department - Course:** One Department offers many Courses (1:N).
- **User - Document:** One User (Student/Instructor) can upload many Documents (1:N).
- **Course - Document**: One Course acts as a repository for many Documents (1:N).
- **Document - RankingResult**: Each Document undergoes a verification process resulting in one Ranking score (1:1).
- **User - Flag:** One User (Student) can report many documents, creating multiple Flags (1:N).
- **Document - Flag:** One Document can be flagged multiple times by different users for review (1:N).

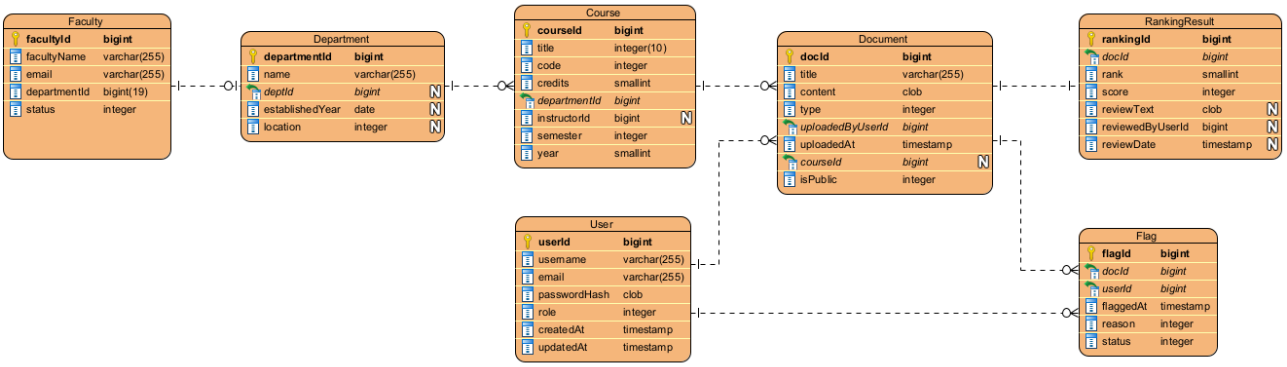


Figure 3.2. *ER Data Model representing the database structure of CampusNote Pro.*

3.4. Problem Domain Class/Object Models

This class diagram represents the main problem domain classes of the CampusNote Pro system. It describes the static structure from an analysis perspective, focusing on real-world concepts and behaviors without implementation details .

Problem Domain Classes:

1. **Student:**
   - **Attributes:** +userId, +coinBalance, +rank
   - **Operations:** +uploadDocument(), +downloadDocument(), +searchKeyword(), +reportDocument()
2. **Admin:**
   - **Attributes:** +adminId, +role

- **Operations:** +manageUserAccount(), +reviewFlaggedContent(), +deleteDocument()

3. **Document:**
   - **Attributes:** +docId, +title, +status, +qualityScore
   - **Operations:** +updateStatus(), +calculateFinalRank()
4. **Course:**
   - **Attributes:** +courseCode, +courseName
   - **Operations:** +listDocuments(), +filterByDifficulty()
5. **RankingService (System Class):**
   - **Attributes:** +serviceStatus, +passingThreshold
   - **Operations:** +evaluateQuality(), +extractKeywords()
6. **Flag:**
   - **Attributes:** +flagId, +reason, +status
   - **Operations:** +createFlag(), +resolveFlag()
   -

Relationships and Multiplicities:

- **Student "1" --> "0..*" Document:** uploads academic materials.
- **Course "1" --> "0..*" Document:** categorizes materials for students.
- **RankingService "1" --> "0..*" Document:** verifies quality and assigns scores.
- **Student "1" --> "0..*" Flag:** reports inappropriate or low-quality content.
- **Document "1" --> "0..*" Flag:** contains reports awaiting moderation.
- **Admin "1" --> "0..*" Flag:** reviews and resolves moderation tasks.

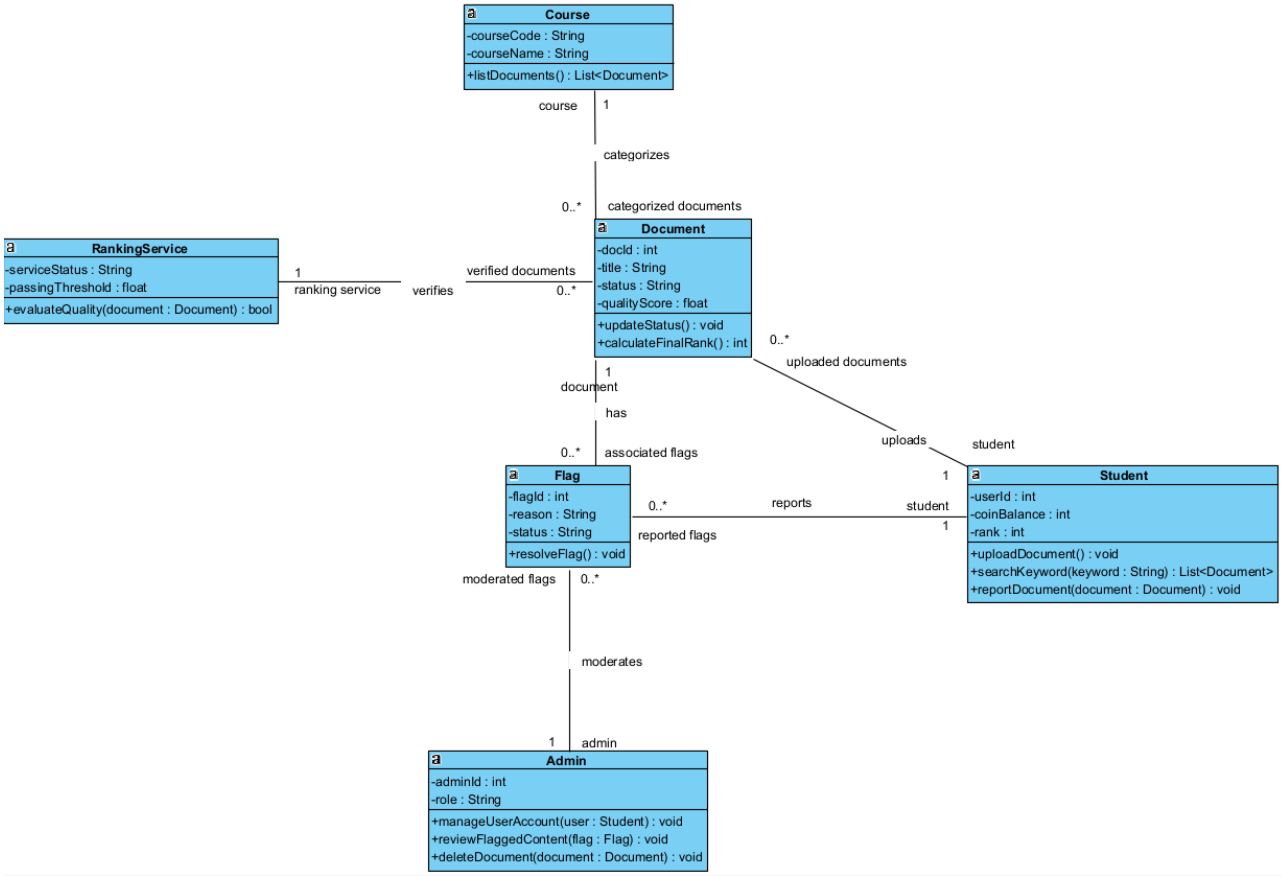


Figure 3.3. *The class diagram of CampusNote Pro.*

3.5. Dynamic Models

Dynamic models describe the dynamic behavior of the system, showing how it works over time and how different parts interact. This activity diagram illustrates the core workflow of the gamified document verification process in the CampusNote Pro system. It details the step-by-step sequence starting from a student uploading a draft, through the external AI Ranking Service's evaluation, and concluding with the conditional logic to either publish the note and award CampusCoins or reject the file.

Scenario: Successful Gamified Document Upload and Rank Progression
Activity Flow

1. Start
2. Student logs into CampusNote Pro.
3. Student navigates to the target Faculty and Course Code.
4. Student uploads the PDF study note.
5. System saves the document in "Draft" status.
6. AI Ranking Service evaluates the document quality.
7. Decision: Check Quality Score
    - *[Branch: score > 80]* System changes document status to "Published".
    - *(Note: If score <= 80, the system rejects the document, notifies the user, and the flow ends).*
8. System awards CampusCoins to the student's balance.
9. System evaluates the new coin balance against rank thresholds.
10. Decision: Check Rank Threshold
    - *[Branch: balance >= threshold]* System upgrades the student's virtual rank.
    - *(Note: If balance < threshold, the flow directly ends).*
11. End

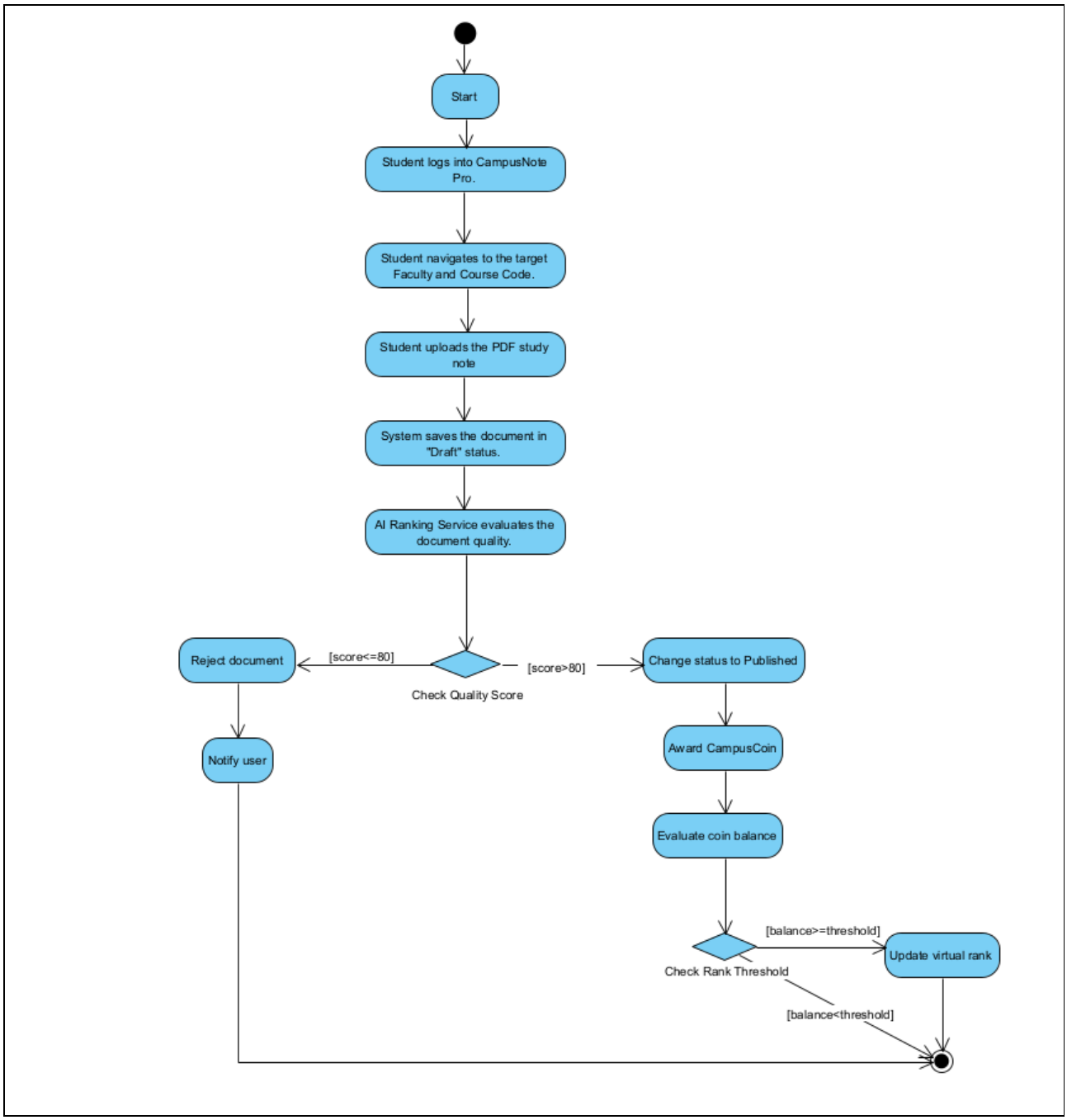


Figure 3.4. *The activity diagram of the CampusNote Pro document verification and gamification process.*

This sequence diagram illustrates the dynamic object interactions during the document upload and verification process. It details the sequential flow of synchronous and asynchronous messages between the Student, Web UI, Backend API, Database, and the external AI Service, including the alternative condition (alt) block for successful publication or rejection based on the AI quality score.

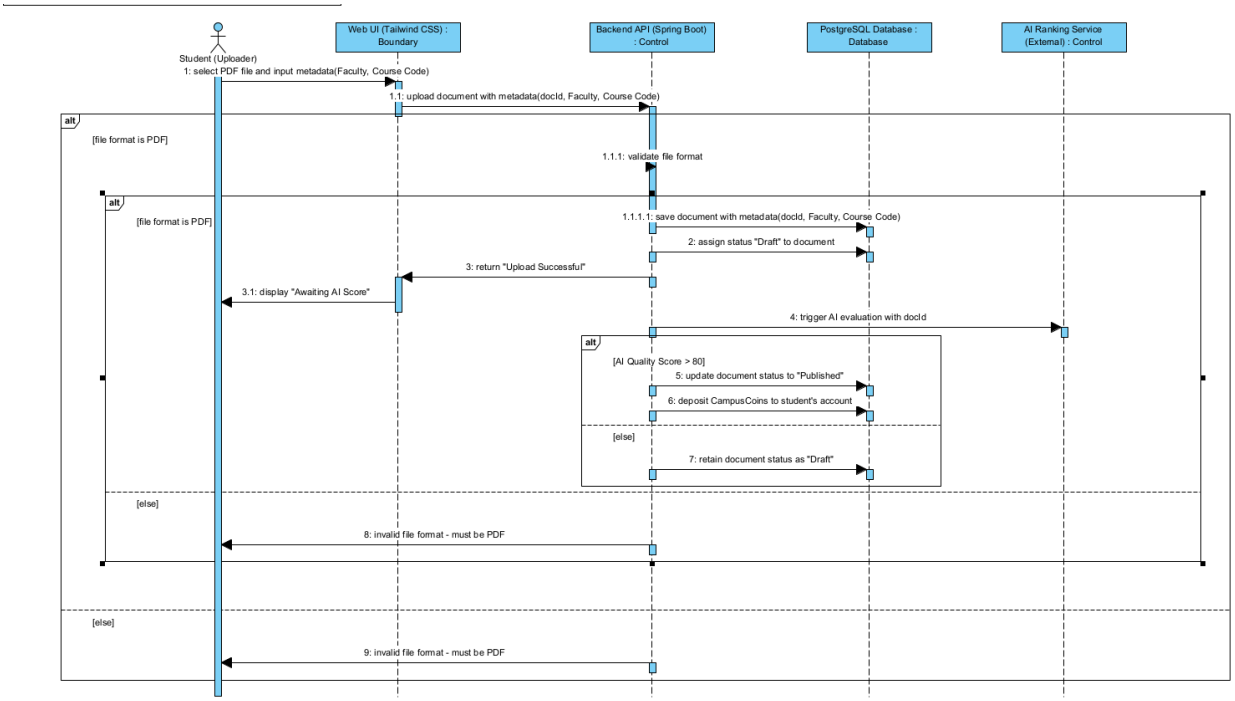


Figure 3.5. *The sequence diagram.*

3.6. Component, Package and Deployment Models

This deployment diagram shows the physical execution environments and hardware nodes of the CampusNote Pro platform. It illustrates how the Student PC connects to the Heroku Cloud Platform via HTTPS, which hosts the Spring Boot application and PostgreSQL database, while securely communicating with a separate server running the external AI Engine.

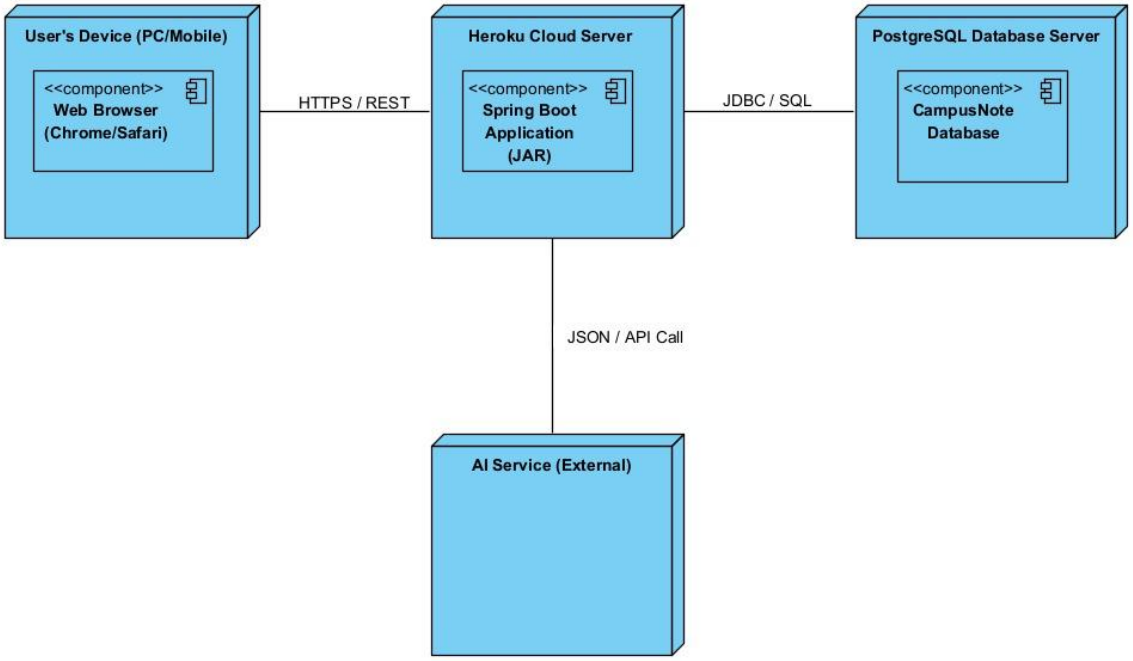


Figure 3.7. The Deployment Diagram of CampusNote Pro.

The package diagram demonstrates the logical grouping of the system's classes into distinct modules. It highlights the separation of concerns by dividing the system into UI, API, Security, and Database packages, ensuring a modular and maintainable codebase.

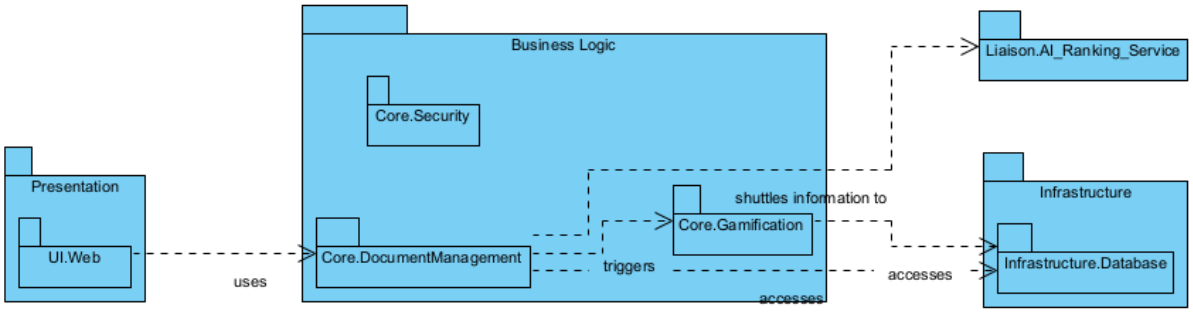


***Figure 3.8**. The Package Diagram of CampusNote Pro.*

This component diagram illustrates the main software modules of the system and their dependencies. It highlights how the Web UI component interacts with the central Backend API component, which in turn orchestrates data exchange with the external AI component for document evaluation and the Database component for storage.

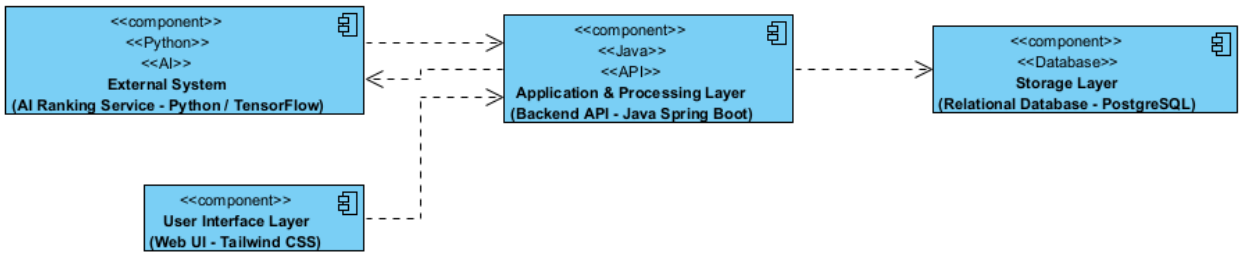


Figure 3.9. *The component diagram of CampusNote Pro.*

3.7. System Architecture

This system architecture diagram presents the high-level layered structure of CampusNote Pro. The architecture is organized into four main layers: the User Interface Layer (Tailwind CSS), the Application & Processing Layer (Java Spring Boot), the Storage Layer (PostgreSQL), and an External System running the AI Ranking Service (Python). It illustrates the clear flow of data, such as document uploads, AI scoring requests, and persistent data storage operations across these isolated layers.

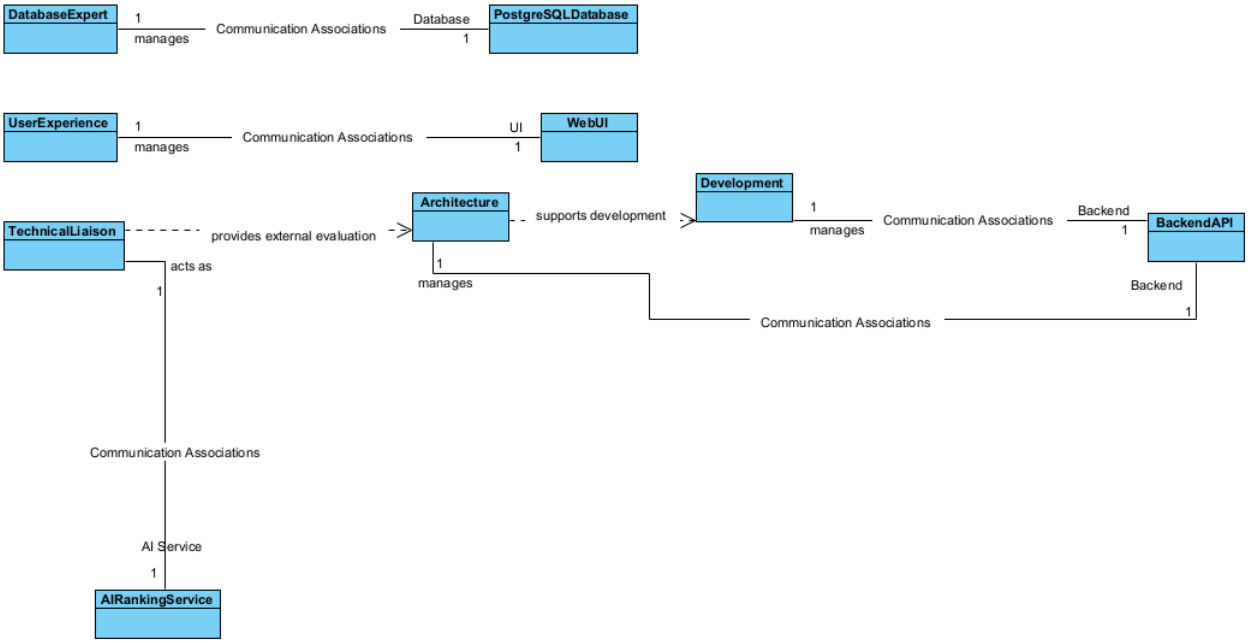
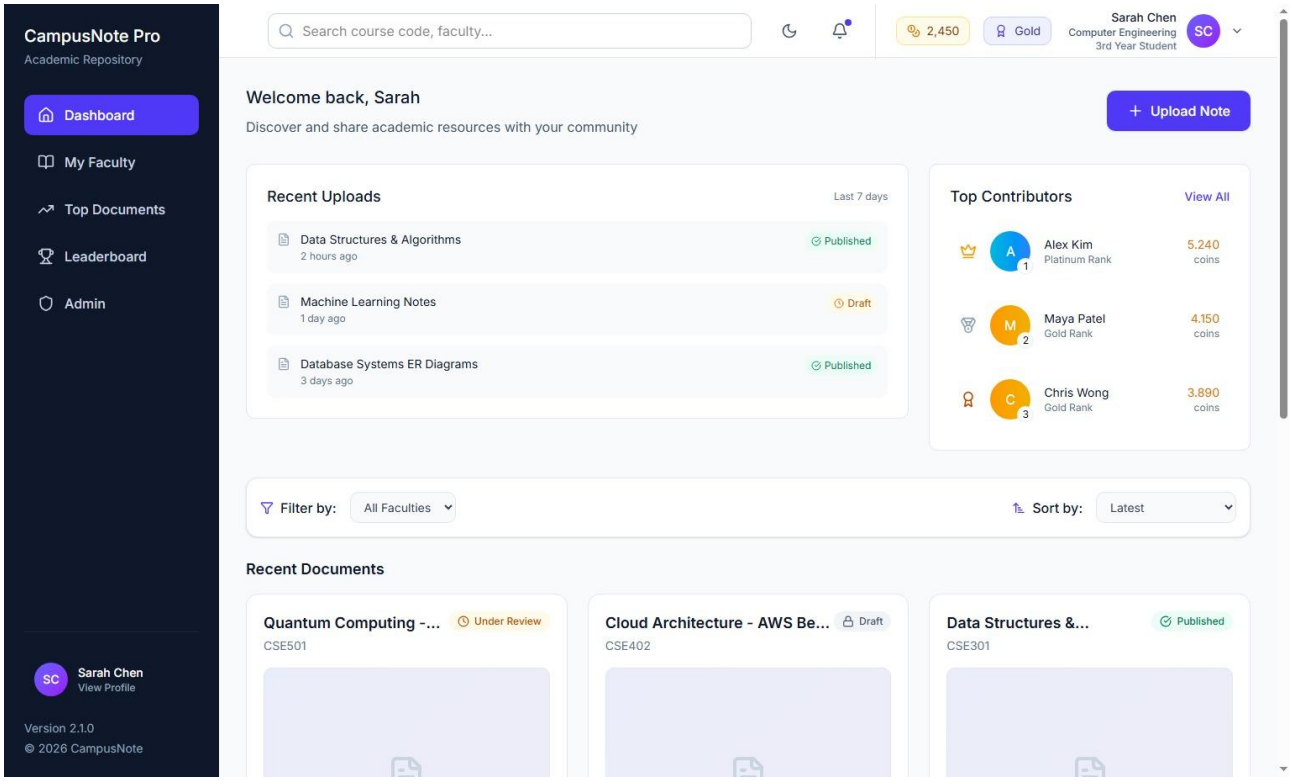
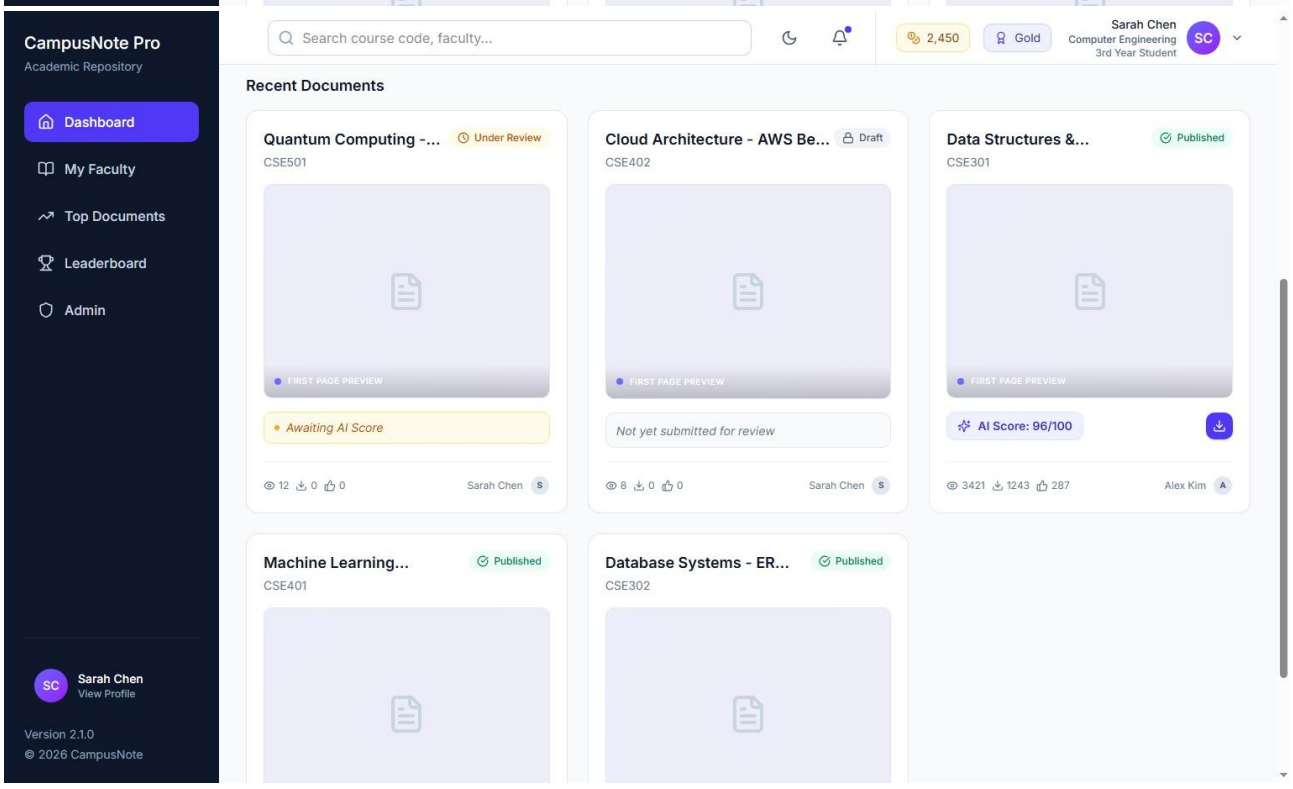


Figure 3.6*. The high-level layered system architecture diagram of CampusNote Pro.*

3.8. User Interfaces





Purpose: This screen serves as the primary central hub for students to discover academic resources, track their gamification progress, and monitor the latest community contributions.

Information Displayed: It presents the user's current CampusCoin balance and virtual rank in the top navigation. The main content area features a "Top Contributors" leaderboard, a list of recent uploads, and detailed document cards. These cards display a first-page thumbnail preview , course codes, academic engagement metrics (views/downloads), and distinctly color-coded verification statuses such as Draft and Published. For "Published" notes, the AI Quality Score is also visibly displayed.

User Actions: Users can utilize the top search bar to find specific keywords, apply Faculty filters, and sort the document feed. They can also initiate a new document submission via the "Upload Note" button , or click thedistinct download icon on verified "Published" documents to retrieve study materials.

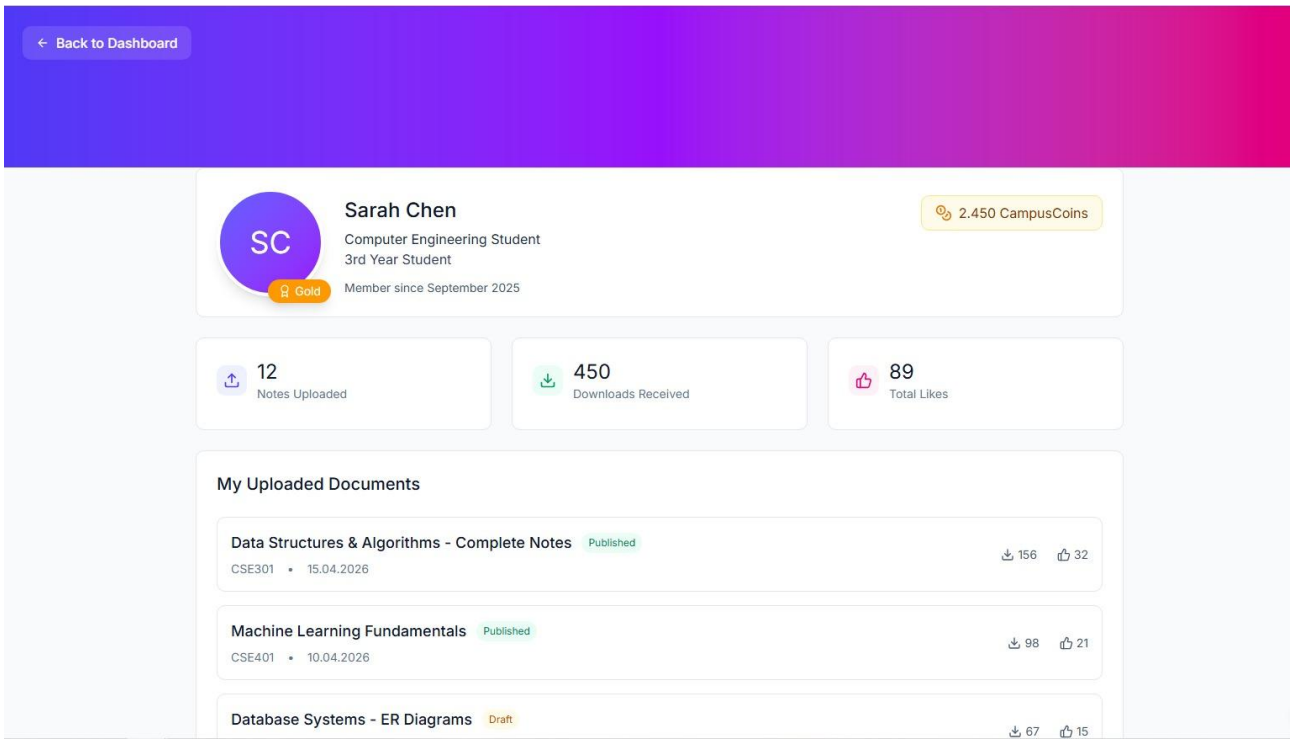


Figure 3.11. *User Profile Screen*

- **Purpose:** This screen provides a personalized overview of a student's academic contributions and their gamification progress within the platform.
- **Information Displayed:** It displays the user's current CampusCoin balance, rank badge (e.g., Gold), overall engagement statistics (total notes uploaded, downloads received, likes), and a detailed history of their uploaded documents.
- **User Actions:** The user can monitor the verification status of their submitted documents, track the engagement metrics of their published notes, and navigate back to the main dashboard.

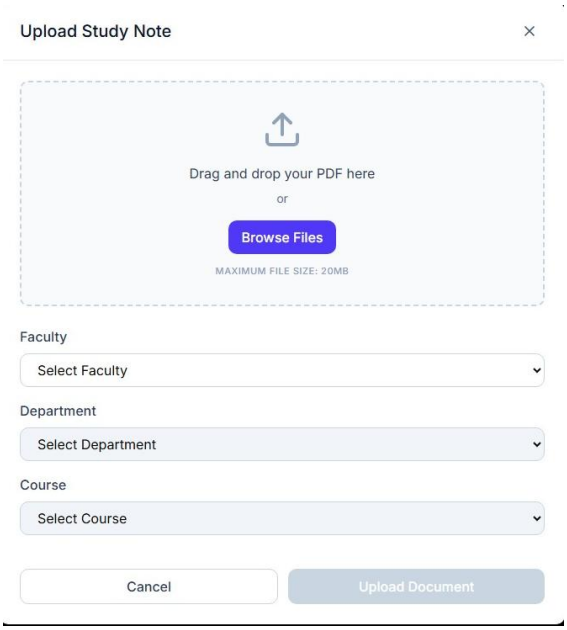
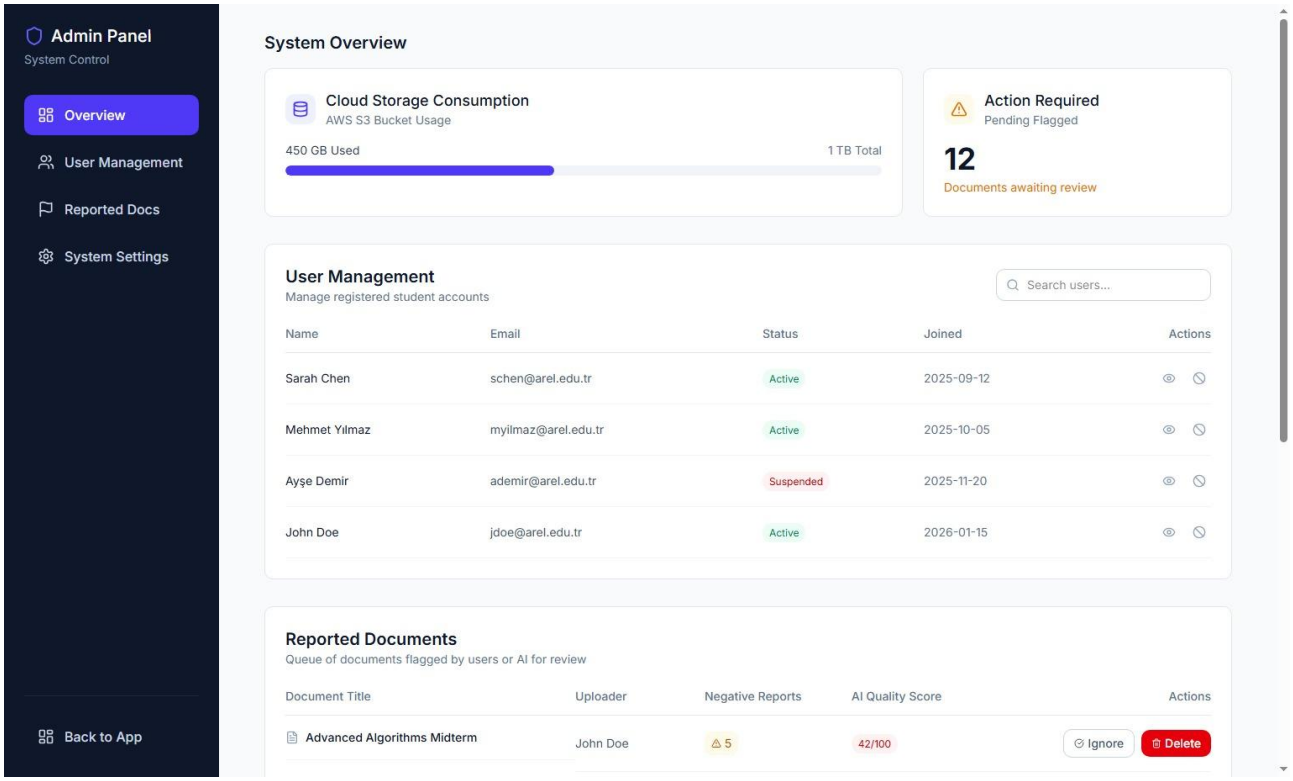


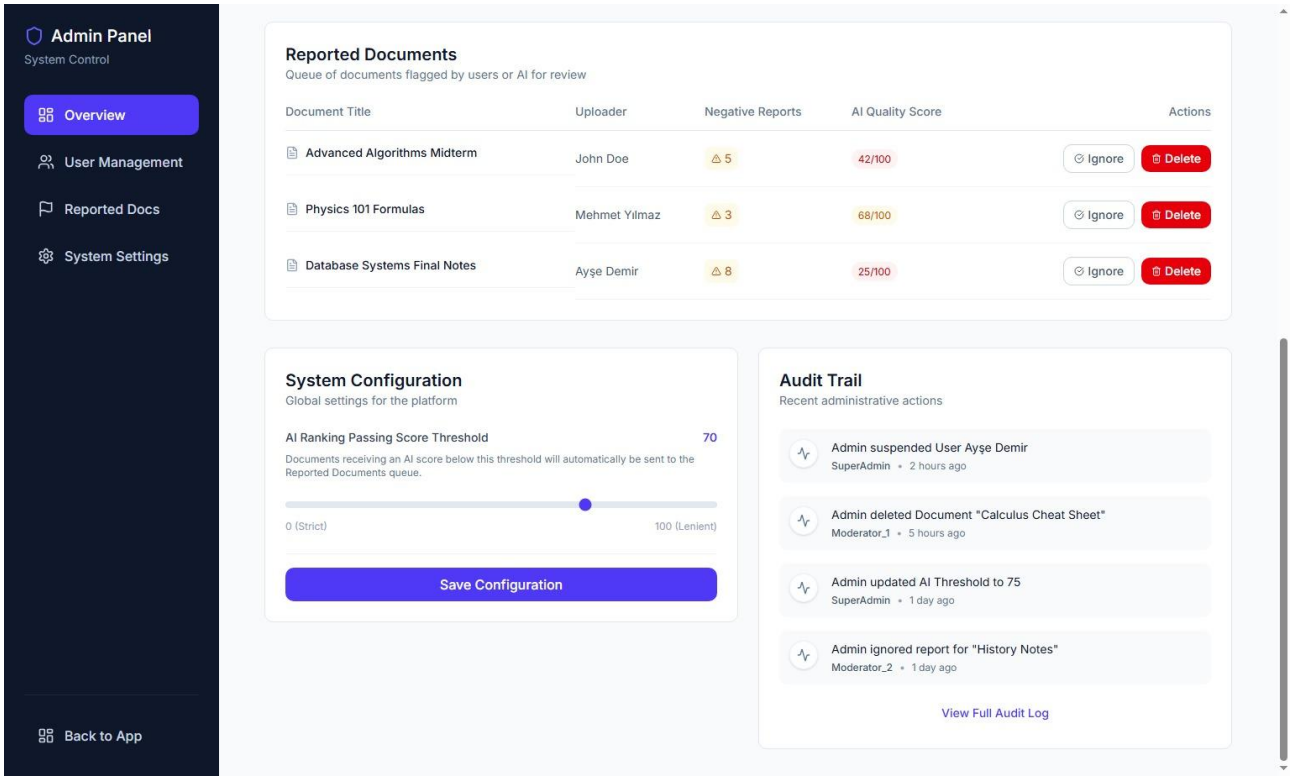
Figure 3.12. Document Upload Screen *[Buraya ui-file.jpeg görselini yapıştırın]*

Purpose: This interface allows students to submit their academic study notes to the system for AI verification.

Information Displayed: It features a drag-and-drop file upload area specifically indicating the required PDF format and a clear helper text stating the maximum file size limit of 20MB. Additionally, it presents hierarchical dropdown menus for precise academic categorization.

User Actions: The user can browse or drag a PDF file into the dropzone, strictly categorize the document by selecting the target Faculty, Department, and Course, and click the "Upload document" button to send the file into the "Draft" status for subsequent AI evaluation.





Purpose: This screen serves as a restricted, centralized control panel for authorized system administrators to manage student accounts, moderate flagged academic content, and configure core platform settings.

Information Displayed: The dashboard is divided into several key data widgets:

- **System Overview:** Displays real-time Cloud Storage Consumption and an alert for the number of pending flagged documents awaiting review.
- **User Management:** A data table listing registered student accounts (using @arel.edu.tr emails), their registration dates, and their current account status (e.g., Active, Suspended).
- **Reported Documents:** A queue of documents flagged by users or the AI, showing the document title, uploader, number of negative reports received, and the AI Quality Score.
- **System Configuration & Audit Trail:** Shows the current "AI Ranking Passing Score Threshold" alongside a read-only list of recent administrative actions (e.g., document deletions, user suspensions) to ensure traceability.

User Actions: Administrators can search for specific users and temporarily suspend their accounts. In the moderation queue, they can evaluate flagged items and choose to either "Delete" the document (for academic integrity violations) or "Ignore" the report. Additionally, admins can use the slider to adjust the global AI passing score threshold and save the new configuration.

3.9. Minimum Requirements

To satisfy the core objectives of the CampusNote Pro platform within the 10-week development timeline, the following elements constitute the mandatory minimum project scope:

3.9.1. Modeling Deliverables (The Analysis MVP)

Per the professor’s guidelines, the system analysis must be supported by a specific minimum set of UML artifacts:

- **Use Case Models**: One comprehensive Use Case Diagram and a minimum of four detailed textual descriptions covering Authentication, Upload, AI Verification, and Search/Retrieval.
- **Data Model**: One Entity-Relationship (ER) Diagram representing the relational persistence layer in 3NF, including User, Faculty, Course, and Document entities.
- **Dynamic Model**: One Activity Diagram illustrating the "Draft-to-Published" state machine and one Sequence Diagram detailing the REST API interaction with the external AI service.
- **System Architecture**: One high-level Layered Architecture diagram showing the client-server boundaries between the Tailwind CSS frontend and Spring Boot backend.
- **User Interfaces**: At least three high-fidelity UI mockups (Dashboard, Upload Modal, and Admin Panel).

3.9.2. Core Functional MVP (The Software MVP)

The implementation phase must deliver these high-priority (Must Have) behaviors to be considered successful:

- **Academic Hierarchy**: A working directory structure that categorizes PDFs strictly by Faculty and Course Code.
- **AI-Driven Lifecycle**: Automated transition of documents from "Draft" status to "Published" based on a score received from the Python-based AI service.

- **Gamified Incentives**: Immediate distribution of **CampusCoins** to user accounts upon successful document publication.
- **Search & Retrieval**: A keyword-based search engine that exclusively displays verified, high-quality materials.

3.9.3. Technical Constraints

The final product must adhere to these environmental requirements for the Phase 2 audit:

- **Deployment**: A functional SaaS application hosted on the **Heroku** cloud platform.
- **Stack**: Backend developed in **Java Spring Boot** with data persistence in **PostgreSQL**.
- **Security**: All user passwords must be hashed using **BCrypt** before database entry.

4. REQUIREMENTS VALIDATION

Requirements validation is performed after requirements elicitation and analysis to ensure that the defined requirements for CampusNote Pro are correct, complete, verifiable (testable), and ready for the Agile implementation phase.

4.1. Validation Approach

- The requirements for CampusNote Pro were validated using the following methods:
    - **Team Reviews (Scrum Refinement):** All team members (Project Leader, Developers, and Tester) reviewed the functional and non-functional requirements to ensure alignment with the system architecture.
    - **Testability Check (BDD/TDD Alignment):** As part of the QA process, every functional requirement was evaluated to ensure it could be translated into executable JUnit and JBehave test scenarios.
    - **Proxy Stakeholder Evaluation:** The team acted as proxy students (uploaders and consumers) to verify that the requirements solve the targeted "Information Silo" and "Temporal Decay" problems.

Validation Results

1. **Correctness**
    - The requirements directly address the core problem of academic information loss at Istanbul Arel University.
2. **Completeness**
    - All essential phases of the document lifecycle are covered:
    - Upload & Categorization (Draft status)
    - AI Verification (Ranking algorithm)
    - Publication & Rewarding (Published status)
    - Search & Download
    - Edge cases and alternative flows are properly defined in the Use Cases.
3. **Consistency**
    - The academic hierarchy is consistently maintained across Functional Requirements, ER Data Models, and Use Cases.
    - Terminology is used uniformly throughout the documentation without contradiction.
4. **Clarity**
    - Requirements are written unambiguously.

- Non-Functional Requirements (NFRs) use strict, measurable metrics rather than vague adjectives.

5. **Realism**
   - The technology stack and deployment environment (Heroku) are fully feasible for the 10-week development timeline.
   - By excluding native mobile app development (CON-TECH-05) and focusing on a responsive web UI, the project scope remains realistic and manageable.
6. **Traceability**
   - Every requirement possesses a unique identifier (e.g., FR-UC-05, NFR-DOC-04).
   - All 'UC' tagged functional requirements are explicitly traced to their fulfilling Use Cases to ensure 100% technical verification.

4.2. Acceptance Criteria

- The following table defines the measurable conditions that must be satisfied for the core requirements of CampusNote Pro to be accepted as complete.

Table 4.1. Acceptance criteria for core requirements.

The requirements for CampusNote Pro have been rigorously validated. They are confirmed to be correct, complete, consistent, and highly testable. The clear definition of measurable Acceptance Criteria ensures that the Quality Assurance (QA) team can seamlessly proceed with writing automated BDD/TDD test suites, providing a solid, verified foundation for the Agile Scrum implementation phase.

4.2.1 User Stories

User Story ID	User Story
US-01	As a student, I want to register using my Arel university email and academic information so that my account is tied to the correct faculty, department, and year.
US-02	As a registered user, I want to log in and log out securely so that only valid active accounts can access the system.
US-03	As a registered user, I want to request a password reset link so that I can regain access when I forget my password.
US-04	As a student, I want to see my department, academic year, rank, and CampusCoin balance so that the dashboard reflects my academic identity and progress.
US-05	As a student uploader, I want to see my uploaded documents and their statuses so that I can track my contribution history.
US-06	As a student uploader, I want to upload a PDF and categorize it by faculty, department, and course so that other students can later find it in the correct academic context.
US-07	As a student uploader, I want department and course options to depend on my selected faculty and department so that documents are categorized consistently.
US-08	As the system, I want Liaison AI to review each draft PDF so that only academically useful notes become visible to students.
US-09	As a student uploader, I want to earn CampusCoins when my note passes verification so that high-quality sharing is rewarded.
US-10	As a student, I want to see a ranked leaderboard so that I can compare contribution progress with other students.
US-11	As a student consumer, I want to search and filter verified notes so that I can quickly find useful material for a course or exam.

US-12	As a student consumer, I want to preview and download a verified PDF so that I can inspect the note before saving it.
US-13	As a student consumer, I want to like useful published notes and report problematic notes so that quality signals improve the repository.
US-14	As an administrator, I want admin pages and APIs protected by role so that students cannot perform moderation or account-management actions.
US-15	As an administrator, I want to view and suspend student accounts so that misuse can be controlled.
US-16	As an administrator, I want to review flagged documents, delete invalid content, or dismiss reports so that the public repository stays trustworthy.
US-17	As an administrator, I want to adjust the AI passing threshold and see audit history so that moderation policy changes are transparent.
US-18	As an administrator, I want to create, update, and delete courses so that upload categorization uses the current academic structure.

4.2.2 BDD Acceptance Criteria

BDD ID	Scenario	Related Requirement(s)	Given	When	Then
BDD-01	Register with valid university identity	FR-ST-01, FR-ST-06, FR-ST-07, FR-ST-08, NFR-ST-67	A student provides full name, password, academic information, and an email ending with @arel.edu.tr	The registration form is submitted	The system creates the account, stores academic metadata, and saves the password as a BCrypt hash
BDD-02	Reject invalid registration email	FR-ST-01	A student provides an email that does not end with @arel.edu.tr	The registration form is submitted	The system rejects the registration and displays an error message
BDD-03	Log in successfully	FR-ST-01, FR-ST-02, FR-ST-03	An active registered user enters correct credentials	The login form is submitted	The system creates an authenticated session and redirects the user to the correct dashboard
BDD-04	Prevent suspended user login	FR-ST-51	A suspended user account exists	The user submits valid credentials	The system rejects the login and explains that the account is suspended
BDD-05	Reset password with valid token	FR-ST-04, NFR-ST-67	A registered user has a valid unexpired reset token	The user submits a new password	The system stores the new password as a BCrypt hash and clears the reset token
BDD-06	Display student dashboard/profile data	FR-ST-06, FR-ST-07, FR-ST-08, FR-ST-31, FR-ST-32	A student is authenticated	The dashboard or profile page loads	The system displays department, academic year, CampusCoin balance, rank, and profile details
BDD-07	Show upload history and analytics	FR-ST-09, FR-ST-10, FR-ST-11, FR-ST-12, FR-ST-13	A student has uploaded one or more documents	The profile page loads	The system lists uploaded documents with statuses and shows total uploads, downloads, and likes
BDD-08	Upload valid PDF study note	FR-ST-15, FR-ST-16, FR-ST-19, FR-ST-20, FR-ST-21, FR-ST-22, FR-DOC-17, FR-DOC-18	A student selects a PDF under 20MB and provides required course metadata	The upload form is submitted	The system stores the file, creates a Draft document record, and triggers AI review
BDD-09	Reject invalid upload file	FR-ST-15, FR-ST-16, FR-ST-21	A student selects a non-PDF file or a	The student attempts to upload the file	The system rejects the upload and displays the relevant validation message

			PDF larger than 20MB		
BDD-10	Filter academic hierarchy	FR-DOC-17, FR-DOC-18, FR-ST-19	A student selects faculty and department information	The course dropdown is opened	The system shows only courses belonging to the selected academic hierarchy
BDD-11	Publish high-quality document	FR-ST-23, FR-ST-24, FR-ST-25, FR-ST-26, FR-ST-27, FR-DOC-28, NFR-ST-71	A Draft document is submitted to Liaison AI	The AI score is greater than or equal to the configured threshold	The system changes the document status to Published and makes it visible in feed/search/download
BDD-12	Flag low-quality document	FR-ST-23, FR-ST-24, FR-ST-25, FR-ST-27, FR-DOC-29, FR-ST-30	A Draft document is submitted to Liaison AI	The AI score is below the configured threshold	The system changes the document status to Flagged and sends it to the admin moderation queue
BDD-13	Award CampusCoins after publication	FR-ST-31, FR-ST-33, FR-ST-34, FR-ST-35, FR-ST-36	A student's uploaded document becomes Published	The reward is calculated using course AKTS	The system adds CampusCoins to the student balance and updates rank if needed
BDD-14	Search and filter published notes	FR-ST-39, FR-ST-40, FR-ST-41, FR-ST-42, NFR-ST-70	Published and non-published documents exist	A student searches by keyword, filter, or sorting option	The system returns only Published documents matching the search/filter criteria within the expected response time
BDD-15	Preview and download published PDF	FR-ST-43, FR-ST-44, FR-ST-45, FR-ST-46, NFR-ST-69	A Published document exists	A student previews or downloads the document	The system displays the preview, serves the PDF for download, and updates view/download counters
BDD-16	Like and report document	FR-ST-37, FR-ST-57	A student is authenticated and a Published document exists	The student likes or reports the document	The system updates the like/report count and flags the document after five negative reports
BDD-17	Restrict admin features	FR-ST-47	A non-admin user attempts to access an admin page or API	The request is submitted	The system denies access and prevents administrative actions
BDD-18	Manage users and moderation	FR-ST-48, FR-ST-49, FR-ST-50, FR-ST-51, FR-ST-52, FR-ST-55, FR-ST-56, FR-ST-58, FR-ST-59, FR-ST-61	An admin is authenticated	The admin suspends a user, deletes a flagged document, or dismisses a report	The system performs the action, updates the affected record, and writes an audit log
BDD-19	Configure AI threshold and audit trail	FR-ST-60, FR-ST-62, FR-ST-63, FR-DOC-64	An admin is authenticated	The admin changes the AI threshold or opens the audit/system status page	The system stores the new threshold, applies it to future AI reviews, and displays audit/storage information
BDD-20	Manage academic course catalog	FR-DOC-17, FR-DOC-18, FR-ST-19, ASM-DOC-79	An admin is authenticated and academic metadata exists	The admin creates, updates, or deletes a course	The system updates the course catalog and reflects the change in upload categorization

4.2.3 Acceptance Criteria

User Story	Related Requirements	Acceptance Criteria Summary
US-01 Register With University Identity	FR-ST-01, FR-ST-06, FR-ST-07, FR-ST-08, NFR-ST-67	The system accepts registration only with a valid Arel university email, stores academic metadata, rejects invalid or duplicate emails, and saves the password securely as a BCrypt hash.
US-02 Log In and End Session	FR-ST-01, FR-ST-02, FR-ST-03, FR-ST-05, FR-ST-51	The system allows active registered users to log in, rejects invalid credentials or suspended accounts, displays clear error messages, and invalidates the session after logout.
US-03 Recover Password	FR-ST-04, NFR-ST-67	The system generates a valid reset token for registered users, allows password reset with an unexpired token, rejects invalid tokens, and stores the new password as a BCrypt hash.
US-04 View Academic Profile and Dashboard Identity	FR-ST-06, FR-ST-07, FR-ST-08, FR-ST-31, FR-ST-32	The dashboard/profile displays the student's department, academic year, CampusCoin balance, virtual rank, and registration information where available.
US-05 View Upload History and Analytics	FR-ST-09, FR-ST-10, FR-ST-11, FR-ST-12, FR-ST-13	The profile page lists the student's uploaded documents with current status and correctly summarizes total uploads, downloads, and likes.
US-06 Upload a Valid PDF Study Note	FR-ST-15, FR-ST-16, FR-ST-19, FR-ST-20, FR-ST-21, FR-ST-22, FR-DOC-17, FR-DOC-18	The system accepts only valid PDF files under 20MB with required course metadata, creates the document in Draft status, and triggers AI review after upload.
US-07 Select Academic Hierarchy	FR-DOC-17, FR-DOC-18, FR-ST-19	The upload form filters departments and courses according to the selected faculty/department and requires a valid course before submission.
US-08 Automatically Review Uploaded Notes	FR-ST-23, FR-ST-24, FR-ST-25, FR-ST-26, FR-ST-27, FR-DOC-28, FR-DOC-29, FR-ST-30, NFR-ST-66, NFR-ST-71	Liaison AI reviews uploaded drafts, extracts text, assigns a quality score, publishes documents meeting the threshold, and flags low-quality documents for moderation.
US-09 Award CampusCoins for Published Notes	FR-ST-31, FR-ST-33, FR-ST-34, FR-ST-35, FR-ST-36	The system awards CampusCoins only when a document is published, calculates the reward from course AKTS, updates the user balance, and refreshes the virtual rank.
US-10 View Leaderboard	FR-ST-38	The leaderboard displays users ordered by CampusCoin balance and supports faculty-based ranking where applicable.
US-11 Search and Filter Published Notes	FR-ST-39, FR-ST-40, FR-ST-41, FR-ST-42, NFR-ST-70	Students can search published notes by keyword, filter results by faculty, sort by download count, and receive results within the expected response time.
US-12 Preview and Download a Published PDF	FR-ST-43, FR-ST-44, FR-ST-45, FR-ST-46, NFR-ST-69	Published documents can be previewed and downloaded, thumbnails are generated, view/download counters are updated, and non-published documents are protected from download.

US-13 Like and Report a Document	FR-ST-37, FR-ST-57	Students can like published documents, toggle existing likes, cannot like non-published documents, and the system flags a document after five negative reports.
US-14 Restrict Admin Dashboard Access	FR-ST-47	Admin pages and admin API operations are accessible only to authenticated admin users; student users are denied access.
US-15 Manage User Accounts	FR-ST-48, FR-ST-49, FR-ST-50, FR-ST-51, FR-ST-59	Admins can view and search registered users, suspend accounts, prevent suspended users from logging in, and generate audit logs for suspension actions.
US-16 Moderate Flagged Documents	FR-ST-52, FR-ST-53, FR-ST-54, FR-ST-55, FR-ST-56, FR-ST-58, FR-ST-61	Admins can view flagged documents, inspect moderation details, delete invalid documents, dismiss reports, republish acceptable documents, and record audit logs.
US-17 Configure AI Threshold and Review Audit Trail	FR-ST-60, FR-ST-62, FR-ST-63, FR-DOC-64	Admins can update the global AI passing threshold, view chronological audit logs, and monitor storage usage statistics.
US-18 Manage Academic Course Catalog	FR-DOC-17, FR-DOC-18, FR-ST-19, ASM-DOC-79	Admins can create, update, and delete course records so that upload categorization remains aligned with the current academic structure.

5. TRACEABILITY

The Traceability Matrix ensures that every high-level student requirement for CampusNote Pro is reflected in the system design and verified through rigorous testing. This alignment prevents "requirement leakage" and guarantees that the final platform delivered on Heroku meets 100% of the initial project goals.

The following matrices establish a bi-directional link between our elicitation artifacts and our Quality Assurance (QA) suite, providing a comprehensive representative sample of the system's core subsystems. For each requirement, it identifies:

- **Use Case:** The behavioral model defined in Section 3.
- **Component / Module:** The specific part of the Java/Spring Boot backend or Tailwind CSS frontend responsible for the feature.
- **Test Case:** The specific JUnit (TDD) or JBehave (BDD) test suite that validates the functionality.

Traceability Matrix

Table 5.1. Functional traceability matrix.

Requirement ID	Description	Associated Use Case	Component / Module	BDD / Automated Test Case
FR-ST-01	University Email Auth	UC-01 Authenticate User	Auth Module (Spring Security)	*testRejectNonArelEmailDomains()*
FR-ST-11	Upload History Display	UC-02 Upload Document	UI.Web & Core.DocManagement	*testFetchUserUploadHistory()*
FR-ST-15	PDF Format Enforcement	UC-02 Upload Document	UI.Web & Core.DocManagement	*testRejectNonPdfUploads()*
FR-ST-27	AI Quality Scoring	UC-03 Verify, Publish & Reward	Liaison.AI_Ranking_Service	*testScoreCalculationLogic()*
FR-DOC-28	AI Publication Rule	UC-03 Verify, Publish & Reward	Document State Machine	*testStatusChangesToPublished()*
FR-ST-34	CampusCoin Distribution	UC-03 Verify, Publish & Reward	Core.Gamification	*testAddCoinsOnSuccessfulPublish()*
FR-ST-40	Verified Search Filter	UC-04 Keyword Search	Core.DocManagement	*testSearchExcludesDrafts()*
FR-ST-44	Secure File Retrieval	UC-04 Keyword Search	Core.DocManagement / AWS S3	*testDownloadGeneratesS3Link()*
FR-ST-50	Admin User Suspension	UC-04 Keyword Search	Core.Security / Admin Panel	*testSuspensionRevokesLogin()*

Non-Functional Traceability Matrix

Table 5.2. Non-functional traceability matrix.

Requirement ID	Description	Component / Infrastructure	Verification Method
NFR-DOC-65	99.9% Operational Uptime	Render Cloud Environment	Continuous Ping/Uptime Monitoring API
NFR-ST-66	100% Verification Integrity	Liaison.AI_Ranking_Service	Automated System Integration Test
NFR-ST-67	BCrypt Password Security	Infrastructure.Database (PostgreSQL)	verifyPasswordIsHashedInDatabase()
NFR-ST-68	Automated Data Backups	Infrastructure.Database (PostgreSQL)	Cron Job Log Audit
NFR-ST-69	Preview Latency (≤ 2.0s)	Core.DocManagement / AWS S3	Apache JMeter Load & Stress Test
NFR-ST-71	AI Engine Throughput	Liaison.AI_Ranking_Service	JMeter Performance Profiling (10MB File)

6. CONCLUSION

In this project, the software requirements for CampusNote Pro, a centralized Software as a Service (SaaS) platform, have been comprehensively analyzed and formally defined. The primary motivation was to solve the critical issue of "Information Silos" and the continuous loss of valuable academic study materials distributed across transient social media groups at Istanbul Arel University. Through a rigorous Requirements Engineering process adhering to SWEBOK v4.0 standards, stakeholder needs were elicited, resulting in a robust system designed to hierarchically archive, verify, and share academic documents.

During the requirements analysis phase, core functionalities such as hierarchical document categorization (Faculty, Department, Course Code), AI-based quality verification, and a gamified reward system (CampusCoins) were established. To ensure structural integrity, various system models—including Use Case descriptions, Entity-Relationship (ER) data models, class diagrams, dynamic behavioral models (specifically Sequence and Activity diagrams), and high-level client-server architectural diagrams—were created using Visual Paradigm. Furthermore, a strict Traceability Matrix was developed to link every functional requirement directly to executable JUnit and JBehave test cases, establishing a highly verifiable foundation for the Quality Assurance (QA) team and the upcoming Agile Scrum implementation.

Key system design decisions included adopting a layered architecture with a RESTful Java/Spring Boot backend, an external Python/TensorFlow service for the AI Ranking Algorithm, a responsive Tailwind CSS frontend, and a PostgreSQL database, all strategically targeted for deployment on the Heroku cloud infrastructure. The system modeling proceeded with necessary assumptions regarding stable internet connectivity and the autonomous capability of the AI ranking algorithm to process documents. To maintain a realistic and achievable 10-week development timeline, the project constraints explicitly excluded native mobile application development, focusing entirely on a mobile-first web interface.

In the future, CampusNote Pro can be further expanded and improved. Potential enhancements include developing dedicated native iOS and Android applications to maximize student accessibility, implementing real-time collaborative document editing tools, and integrating the platform directly with Istanbul Arel University's official Learning Management System (LMS) to synchronize course codes automatically.

7. REFERENCES

[1] IEEE Computer Society, SWEBOK v4.0: Guide to the Software Engineering Body of Knowledge. IEEE, 2024. [Online]. Available: https://www.computer.org/volunteering/boards-and-committees/professional-educational-activities/se-body-of-knowledge

[2] M. Seidl, M. Scholz, C. Huemer, and G. Kappel, UML @ Classroom: An Introduction to Object-Oriented Modeling, 1st ed. Vienna, Austria: Springer, 2015.

[3] Object Management Group (OMG), "Unified Modeling Language (UML) Specification v2.5.1," Dec. 2017. [Online]. Available: https://www.omg.org/spec/UML/2.5.1/

[4] C. Walls, Spring in Action, 6th ed. Shelter Island, NY, USA: Manning Publications, 2022.

[5] J. M. Hellerstein, M. Stonebraker, and J. Hamilton, "Architecture of a Database System," Foundations and Trends in Databases, vol. 1, no. 2, pp. 141–259, 2007.

[6] A. Wathan, Tailwind CSS: Up and Running.

[7] A. Vaswani et al., "Attention is all you need," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 5998–6008.

8. DESCRIPTIONS, ABBREVIATIONS, AND DICTIONARY

8.1. Descriptions

Table 8.1. Descriptions.

Word or Concept	Description
CampusCoin	A virtual reward unit given to students to encourage the upload of high-quality study notes.
Draft Status	A temporary state for an uploaded document awaiting evaluation by the Artificial Intelligence algorithm.
Published Status	The state of a document that has successfully passed the AI evaluation (scoring 80 or above) and is publicly accessible in the system.
Gamification	A rank and point system integrated into the platform to increase user engagement and participation.
SaaS	(Software as a Service) A software distribution model where the platform is hosted on the cloud (Heroku) and accessed via a web browser.

8.2. English Abbreviations

Table 8.2. English abbreviations.

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
DB	Database
UI	User Interface
READ	Requirements Elicitation and Analysis Document